

SLOT6:

Implement the end-to-end solution integrating Express, Node and MongoDB, MVC

Tóm tắt nội dung

16 Implement the end-to-end solution integrating Express, Node and MongoDB

17 Model - View – Controller

1. Implement the End-to-End Solution Integrating Express, Node và MongoDB

Mục tiêu:

Tạo một ứng dụng web hoàn chỉnh với backend Node.js sử dụng framework Express và kết nối với cơ sở dữ liệu MongoDB.

Các bước chính:

1.1. Cài đặt môi trường

- Cài Node.js và npm.
- Khởi tạo dự án bằng: `npm init`
- Cài các package cần thiết:

```
bash
npm install express mongoose body-parser cors
```

1.2. Tạo server với Express

```
js

const express = require('express');
const app = express();
const port = 3000;

app.use(express.json());

app.listen(port, () => {
  console.log(`Server is running on port ${port}`);
});
```

1.3. Kết nối MongoDB với Mongoose

```
js

const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/myapp', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
```

1.4. Tạo Schema và Model

```
js

const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  name: String,
  email: String
});

const User = mongoose.model('User', UserSchema);
```

1.5. Tạo các API Endpoint (CRUD)

```
js

// GET all users
app.get('/users', async (req, res) => {
  const users = await User.find();
  res.json(users);
});

// POST create a new user
app.post('/users', async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.status(201).send(user);
});
```

2. Mô hình MVC (Model- View- Controller)

Mục tiêu:

Tổ chức mã nguồn sạch sẽ, dễ quản lý bằng cách phân tách các chức năng thành 3 phần:

2.1. Model

- Chứa logic dữ liệu, định nghĩa schema, và tương tác với MongoDB.
- Ví dụ: /models/User.js

```
js

const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  name: String,
  email: String,
});

module.exports = mongoose.model('User', UserSchema);
```

2.2. View

- Dành cho frontend (HTML, EJS, hoặc một frontend framework).
- Nếu là API backend-only thì **không cần View**.
- Nếu có, thường đặt trong thư mục /views.

2.3. Controller

- Xử lý logic của request, tương tác với model và trả về response.
- Ví dụ: /controllers/userController.js

```
js

const User = require('../models/User');

exports.getAllUsers = async (req, res) => {
  const users = await User.find();
  res.json(users);
};

exports.createUser = async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.status(201).send(user);
};
```

2.4. Routes

- Định tuyến các request đến controller tương ứng.
- Ví dụ: /routes/userRoutes.js

```
js
```

```
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');

router.get('/users', userController.getAllUsers);
router.post('/users', userController.createUser);

module.exports = router;
```

Tích hợp tất cả lại trong **app.js** hoặc **server.js**

```
const express = require('express');
const mongoose = require('mongoose');
const userRoutes = require('./routes/userRoutes');

const app = express();
app.use(express.json());
app.use('/api', userRoutes);

mongoose.connect('mongodb://localhost:27017/myapp');

app.listen(3000, () => console.log('Server running on port 3000'));
```

✓ CÁC NỘI DUNG TRỌNG TÂM, CỐT LÕI CẦN LƯU Ý:

🔑 1. Kiến trúc tổng thể ứng dụng Node.js

- Hiểu **luồng xử lý request** → **response** trong ứng dụng:
 - Client gửi request
 - Router phân hướng đến Controller
 - Controller xử lý logic và tương tác Model
 - Model truy vấn DB (MongoDB)
 - Kết quả trả về client (thường là JSON)

🔧 2. Kỹ năng tích hợp các thành phần

Express:

- Tạo server, định nghĩa route
- Middleware (`express.json()`, `cors()`, `logger...`)

MongoDB + Mongoose:

- Hiểu khái niệm **Schema** và **Model**
- Biết cách thao tác với cơ sở dữ liệu qua các phương thức:
 - `find()`, `findById()`, `save()`, `updateOne()`, `deleteOne()`

📦 3. Hiểu và áp dụng mô hình MVC

- **Model:** Quản lý dữ liệu, kết nối DB
- **View:** (nếu có) render HTML hoặc trả JSON (nếu là API)
- **Controller:** Chứa logic xử lý chính của app
- **Route:** Giao tiếp giữa URL và Controller

➡ Lợi ích: Dễ bảo trì, mở rộng, chia module rõ ràng

📁 4. Tổ chức cấu trúc thư mục chuẩn

Một cấu trúc thư mục chuẩn giúp dễ phát triển và quản lý:

```
bash
```

/models → định nghĩa schema và models
/controllers → chứa các hàm xử lý logic
/routes → định nghĩa các API endpoints
/app.js → khởi tạo ứng dụng Express

5. Các thao tác CRUD cơ bản

- **Create:** Tạo mới tài liệu trong MongoDB
- **Read:** Lấy dữ liệu từ MongoDB
- **Update:** Cập nhật tài liệu
- **Delete:** Xóa tài liệu

Đây là nền tảng của hầu hết các ứng dụng backend.

6. Những lỗi thường gặp cần chú ý

- Không kết nối được MongoDB (sai URL hoặc chưa chạy MongoDB)
- Không dùng `express.json()` nên body không được parse
- Lỗi `async/await` không có `try/catch` → dễ crash app
- Không export đúng module (Model/Controller)

Tóm lại – Checklist khi học:

Mục	Bạn đã hiểu chưa?
Express tạo API như thế nào?	☒ / ☒
Kết nối và thao tác với MongoDB qua Mongoose?	☒ / ☒
Tách ứng dụng theo mô hình MVC?	☒ / ☒
Cấu trúc thư mục chuẩn Node.js?	☒ / ☒
Viết và test các route CRUD cơ bản?	☒ / ☒

✓ 1. Mô hình MVC hiện tại có lạc hậu không?

? Trả lời ngắn:

KHÔNG lạc hậu, nhưng **KHÔNG** còn là lựa chọn mặc định duy nhất cho mọi dự án như trước đây.

💡 Giải thích:

- MVC (Model – View – Controller) vẫn rất hữu dụng, đặc biệt với:
 - Ứng dụng web server-side truyền thống (Express, Spring, Laravel...)
 - Dự án nhỏ hoặc vừa, nhiều thao tác CRUD
 - Học viên mới học lập trình backend

Nhưng:

- Trong các hệ thống **frontend/backend tách biệt (React, Vue, mobile app)**:
 - View không còn nằm trong backend → phần "V" trong MVC **không còn ý nghĩa rõ ràng**.
 - Lúc này MVC thường chỉ còn **M (Model)** và **C (Controller)**.

🔄 2. Giải pháp thay thế hoặc mở rộng MVC

✓ Một số mô hình/kiến trúc hiện đại hơn tùy quy mô dự án:

📁 1. MVVM (Model - View - ViewModel)

- Thường dùng trong frontend (Vue.js, Angular, React + Redux).
- Không áp dụng trực tiếp cho Node.js backend, nhưng tương thích khi xây hệ thống fullstack.

🌿 2. Mô hình Layered Architecture / Clean Architecture

Được ưa chuộng trong các dự án Node.js/Express lớn hoặc production

Layered

SCSS

Controller → Service (Business Logic) → Repository (Data Access) → Model

Lợi ích:

- Tách biệt rõ vai trò từng lớp
- Dễ test từng phần (unit test services)
- Dễ mở rộng hệ thống lớn, thêm cache, log, v.v.

Ví dụ thư mục:

bash

```
/controllers  
/services  
/repositories  
/models
```

3. Hexagonal Architecture (Ports & Adapters)

- Dành cho hệ thống phức tạp, microservices, event-driven.
- Logic được tách biệt hoàn toàn khỏi cách giao tiếp (REST, GraphQL, CLI...).

4. Domain-Driven Design (DDD)

- Tách nghiệp vụ theo "bounded context".
- Dành cho hệ thống doanh nghiệp phức tạp.
- Dễ kết hợp với TypeScript, NestJS (có cấu trúc theo DDD rất rõ).

3. Khi nào nên dùng MVC, khi nào nên thay thế?

Tình huống	Gợi ý mô hình
Ứng dụng CRUD đơn giản	✅ MVC
Backend cho frontend SPA (React, Vue...)	✅ MVC (không có View)
Hệ thống lớn, nghiệp vụ phức tạp	Layered / Clean Architecture
Microservices, hệ thống CQRS	Hexagonal, DDD
API tốc độ cao, chia module nhỏ	Modular Layered hoặc Clean Architecture

4. Trong Express/Node.js hiện đại – cách tốt nhất là gì?

✅ Sử dụng Layered Architecture (MVC mở rộng):

bash

```
|— controllers/      ← Xử lý request  
|— services/        ← Business logic  
|— repositories/    ← Giao tiếp với DB (Mongoose)  
|— models/          ← Schema  
|— routes/          ← Định tuyến
```

➔ Được xem là phiên bản "MVC hiện đại", phù hợp nhất với Node.js

✅ KẾT LUẬN

- MVC không lạc hậu, vẫn rất tốt để học và bắt đầu.
- Nhưng với dự án thực tế chuyên nghiệp, MVC nên được mở rộng thành mô hình phân lớp rõ ràng hơn (Service, Repository, DTO...).

- Bạn không cần bỏ MVC, mà **nên nâng cấp từ nó**.

✓ **ỨNG DỤNG THỰC TIỄN:**

✓ 1. Tạo RESTful API cho ứng dụng web hoặc mobile

Thực tế:

- Backend cho website thương mại điện tử, mạng xã hội, hệ thống quản lý (CMS, CRM...)
- API cho app di động (React Native, Flutter...)

Áp dụng:

- Express định nghĩa các endpoint (GET /users, POST /login,...)
- MongoDB lưu trữ dữ liệu người dùng, sản phẩm, đơn hàng
- Mô hình MVC tách rõ logic để dễ bảo trì

✓ 2. Xây dựng hệ thống quản lý (CRUD)

Ví dụ thực tế:

- Quản lý sinh viên, khóa học, nhân viên, kho hàng...
- Dashboard admin để theo dõi dữ liệu

Áp dụng:

- Tạo các bảng dữ liệu và thao tác thêm/sửa/xóa
- Model định nghĩa cấu trúc dữ liệu (Student, Product...)
- Controller xử lý logic như tìm kiếm, lọc, phân trang

✓ 3. Triển khai hệ thống xác thực người dùng (Authentication)

Ví dụ thực tế:

- Đăng ký, đăng nhập, phân quyền admin/user
- Lưu thông tin user trong MongoDB

Áp dụng:

- Tạo routes: /register, /login
- Hash mật khẩu (sử dụng bcrypt)
- Controller kiểm tra thông tin đăng nhập và trả về JWT token

✓ 4. Triển khai dự án thực tế lên cloud (production)

Ví dụ:

- Đưa app lên **Render, Vercel, Heroku, DigitalOcean**
- MongoDB dùng **MongoDB Atlas** (cloud)

Áp dụng:

- Biết cách sử dụng biến môi trường .env
- Kết nối MongoDB online
- Cấu hình để ứng dụng chạy mượt trong môi trường thực

✓ 5. Phát triển nhóm & làm việc chuyên nghiệp

- Khi code theo mô hình **MVC** + cấu trúc chuẩn:
 - Dễ chia việc: người làm controller, người làm model, người làm frontend
 - Dễ test, dễ maintain code

Áp dụng:

- Làm việc nhóm với Git
- Mỗi thành viên có thể phụ trách 1 phần của mô hình MVC

✓ 6. Chuẩn bị cho phỏng vấn và xin việc

- Hầu hết công ty tuyển **Node.js Developer** đều yêu cầu:
 - Biết Express
 - Biết MongoDB
 - Biết tổ chức code theo mô hình MVC

Tóm lại:

Kỹ thuật	Ứng dụng thực tế
Express	Xây REST API, quản lý route
MongoDB + Mongoose	Lưu trữ dữ liệu động, phi quan hệ
MVC	Tổ chức code sạch, dễ phát triển và mở rộng
CRUD API	Giao tiếp giữa frontend và backend
JWT/Auth	Bảo mật hệ thống, đăng nhập người dùng
Triển khai thực tế	Đưa app lên môi trường production

CHI TIẾT MỘT SỐ NỘI DUNG:

PHẦN 1: TỔNG QUAN VỀ KIẾN TRÚC ỨNG DỤNG WEB VỚI NODEJS

1.1. Node.js là gì?

- Node.js là môi trường chạy JavaScript trên server (outside browser).
- Dựa trên engine V8 của Google.
- Ưu điểm:
 - Non-blocking I/O (asynchronous)
 - Phù hợp cho API, microservices, ứng dụng thời gian thực (real-time chat/game)

1.2. Express.js là gì?

- Express là **web framework nhẹ** trên nền Node.js.
- Cung cấp các tính năng:
 - Routing
 - Middleware
 - Xử lý request/response
 - RESTful API dễ dàng

1.3. MongoDB là gì?

- MongoDB là NoSQL database dạng **document-based**, dùng định dạng **JSON-like (BSON)**.
- Không cần schema cố định như SQL → dễ mở rộng, linh hoạt.

1.4. Mongoose là gì?

- Mongoose là một ODM (Object Data Modeling) cho MongoDB và Node.js.
- Cho phép bạn:
 - Xác định schema rõ ràng
 - Tạo model để thao tác DB dễ hơn
 - Hỗ trợ middleware, validation, relationship (population)

PHẦN 2: MÔ HÌNH MVC (Model- View- Controller)

Mục tiêu:

Tách biệt **data (Model)**, **giao diện (View)** và **logic xử lý (Controller)**.

Thành phần	Vai trò	Mô tả
Model	Dữ liệu	Định nghĩa schema, tương tác DB
View	Giao diện	Thường dùng khi có HTML frontend
Controller	Xử lý logic	Nhận request, xử lý và trả response
Route (bổ sung)	Kết nối	Gắn URL với controller

Cấu trúc MVC tiêu chuẩn:

bash

```
/controllers
  userController.js
/models
  User.js
/routes
  userRoutes.js
/views (tuỳ chọn - nếu không phải API thì không cần)
/app.js
```

PHẦN 3: KỸ THUẬT VÀ CÁCH TRIỂN KHAI

3.1. Tạo RESTful API cơ bản (CRUD)

Phương thức	Mục đích	Ví dụ
GET	Lấy dữ liệu	/users
POST	Tạo mới	/users
PUT/PATCH	Cập nhật	/users/:id
DELETE	Xoá	/users/:id

Ví dụ chi tiết:

js

```
// controller/userController.js
exports.getAllUsers = async (req, res) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (err) {
    res.status(500).send('Server Error');
  }
};
```

PHẦN 4: KIẾN THỨC CHUYÊN SÂU VÀ MỞ RỘNG

4.1. Middleware trong Express

Middleware là hàm chạy **trước khi đến route handler**.

Ví dụ:

js

```
app.use((req, res, next) => {
  console.log('Request received');
  next();
});
```

Ứng dụng:

- Xác thực (authentication middleware)
- Log request
- Xử lý lỗi tập trung

4.2. Bảo mật cơ bản: Authentication & Authorization

- Dùng **JWT (JSON Web Token)** để xác thực người dùng:

- Khi đăng nhập, tạo token và gửi lại client
- Sau đó client gửi token mỗi request → server kiểm tra hợp lệ
- Dùng middleware để chặn truy cập trái phép:

js

```
const auth = (req, res, next) => {
  const token = req.header('Authorization');
  // verify token ...
  next();
};
```

4.3. Validation dữ liệu

- Kiểm tra input từ client (bảo mật và tránh lỗi).
- Dùng express-validator hoặc mongoose schema validation.

Ví dụ:

js

```
const UserSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true },
  age: { type: Number, min: 18 }
});
```

4.4. Kết nối MongoDB Atlas (trên cloud)

MongoDB Atlas là dịch vụ MongoDB trên cloud:

- Tạo cluster
- Lấy connection string
- Kết nối trong Node.js:

js

```
mongoose.connect(process.env.MONGO_URI);
```

4.5. Deploy ứng dụng lên Production

- Các nền tảng: **Render, Railway, Heroku, Vercel**
- Dùng biến môi trường .env để bảo mật:

bash

```
PORT=3000
MONGO_URI=your-atlas-url
```

- Dùng dotenv package để load biến này:

js

```
require('dotenv').config();
```

PHẦN 5: BÀI TẬP / DỰ ÁN THỰC TẾ ĐỂ ÁP DỤNG

Đề xuất: Xây dựng "Hệ thống quản lý người dùng"

Tính năng:

- API: đăng ký, đăng nhập, lấy danh sách, cập nhật, xoá người dùng
- Tích hợp JWT
- Kết nối MongoDB Atlas

Bổ sung mở rộng:

- Pagination (phân trang dữ liệu)
- Tìm kiếm (search by name/email)
- Upload ảnh đại diện (sử dụng multer)
- Tích hợp frontend (React, EJS...)

✓ TÓM TẮT TOÀN DIỆN KIẾN THỨC CẦN NẮM

Chủ đề	Nội dung chính
Node.js & Express	Xử lý request/response, middleware, route
MongoDB & Mongoose	Schema, model, query dữ liệu
MVC	Phân tách rõ code, dễ bảo trì
CRUD API	Giao tiếp với client
Middleware	Xác thực, xử lý lỗi
JWT Auth	Login bảo mật
Validation	Bảo vệ dữ liệu đầu vào
Deployment	Triển khai cloud, dùng .env

KINH NGHIỆM THỰC TIỄN, VẤN ĐỀ CẦN LƯU Ý, VÀ CÁC MẸO (TIPS)

1. Populate the Comments with the Users' Information upon Querying

Kinh nghiệm thực tiễn

- **Ứng dụng phổ biến:** Populate rất hữu ích trong các ứng dụng có mối quan hệ giữa các collection, như mạng xã hội (bài viết, bình luận, người dùng), thương mại điện tử (sản phẩm, đánh giá, người mua), hoặc hệ thống quản lý nội dung (bài viết, danh mục, tác giả).
- **Tối ưu hóa dữ liệu trả về:** Trong thực tế, không phải lúc nào bạn cũng cần toàn bộ dữ liệu từ collection được tham chiếu. Ví dụ, khi hiển thị bình luận, bạn thường chỉ cần tên và ảnh đại diện của người dùng, không cần toàn bộ thông tin như email hay mật khẩu.
- **Kết hợp với frontend:** Populate giúp giảm số lần gọi API từ frontend, vì dữ liệu liên quan được trả về trong một lần truy vấn. Điều này cải thiện trải nghiệm người dùng (UX) bằng cách giảm thời gian tải.

Vấn đề cần lưu ý

1. Hiệu suất:

- Populate thực hiện các truy vấn bổ sung tới database, có thể làm chậm ứng dụng nếu sử dụng trên tập dữ liệu lớn hoặc populate nhiều cấp (nested populate).
- **Giải pháp:** Chỉ populate các trường cần thiết bằng cách sử dụng select. Ví dụ: javascript

```
Comment.find().populate('user', 'name avatar').exec();
```

- Nếu có nhiều cấp populate, hãy cân nhắc cấu trúc lại schema để giảm sự phụ thuộc vào các mối quan hệ phức tạp.

2. Dữ liệu không tồn tại:

- Nếu tài liệu được tham chiếu (ví dụ: user) bị xóa, trường populate sẽ trả về null. Điều này có thể gây lỗi ở frontend nếu không xử lý đúng.
- **Giải pháp:** Kiểm tra dữ liệu trả về trước khi hiển thị: javascript

```
if (comment.user) {  
  // Hiển thị thông tin user  
} else {  
  // Hiển thị thông báo "Người dùng không tồn tại"  
}
```

3. Thiết kế schema:

- Populate chỉ hoạt động khi bạn đã định nghĩa đúng ref trong schema. Nếu sai ref (ví dụ: tham chiếu đến một collection không tồn tại), populate sẽ thất bại mà không báo lỗi rõ ràng.
- **Giải pháp:** Kiểm tra kỹ schema trước khi sử dụng populate.

Mẹo (Tips)

1. Chọn lọc trường:

- Luôn sử dụng select để chỉ lấy các trường cần thiết, giảm tải cho database:

```
Comment.find().populate('user', 'name avatar -_id').exec(); // -  
_id loại bỏ trường _id
```

2. Populate nhiều cấp có kiểm soát:

- Khi cần populate nhiều cấp, hãy chia nhỏ truy vấn nếu có thể để dễ debug và tối ưu:

```
const comments = await Comment.find().populate({  
  path: 'user',  
  select: 'name',  
  populate: { path: 'profile', select: 'bio' }  
});
```

3. Tối ưu hóa với index:

- Đảm bảo các trường tham chiếu (ObjectId) có index để tăng tốc độ truy vấn:

```
commentSchema.index({ user: 1 });
```

4. Kiểm tra dữ liệu trước khi populate:

- Trước khi populate, kiểm tra xem collection có dữ liệu hay không để tránh truy vấn không cần thiết:

```
const commentCount = await Comment.countDocuments();  
if (commentCount === 0) return res.json([]);
```

5. Sử dụng lean() để tăng tốc:

- Nếu bạn chỉ cần dữ liệu thô và không cần các phương thức Mongoose, hãy sử dụng .lean() để trả về JSON đơn giản, giảm thời gian xử lý:

```
Comment.find().populate('user', 'name').lean().exec();
```

2. Match with MongoDB Regex

Kinh nghiệm thực tiễn

- **Tìm kiếm linh hoạt:** Regex rất hữu ích trong các ứng dụng cần tìm kiếm văn bản, như tìm kiếm sản phẩm, bài viết, hoặc thông tin khách hàng. Trong thực tế, người dùng thường nhập từ khóa không chính xác (ví dụ: "iphon" thay vì "iPhone"), và regex giúp xử lý các trường hợp này.
- **Kết hợp với các tính năng khác:** Regex thường được sử dụng cùng với các bộ lọc khác (như danh mục, giá) để tạo ra các công cụ tìm kiếm mạnh mẽ.
- **Ứng dụng phổ biến:**
 - Trong thương mại điện tử: Tìm kiếm sản phẩm dựa trên tên hoặc mô tả.
 - Trong CRM: Tìm kiếm khách hàng hoặc liên hệ dựa trên tên, email, hoặc số điện thoại.
 - Trong hệ thống chat: Tìm kiếm tin nhắn hoặc người dùng dựa trên từ khóa.

Vấn đề cần lưu ý

1. Hiệu suất:

- Regex trên tập dữ liệu lớn có thể rất chậm, đặc biệt nếu không có index hoặc sử dụng các mẫu regex phức tạp.
- **Giải pháp:** Sử dụng **text index** của MongoDB cho tìm kiếm văn bản thay vì regex khi có thể.

javascript

```
productSchema.index({ name: 'text', description: 'text' });
Product.find({ $text: { $search: 'iphone' } });
```

- Nếu bắt buộc dùng regex, hãy sử dụng các mẫu cụ thể (ví dụ: ^abc thay vì .*abc.*) để giảm thời gian xử lý.

2. Bảo mật:

- Từ khóa do người dùng nhập có thể chứa các ký tự đặc biệt, gây ra lỗi hoặc hiệu suất kém.
- **Giải pháp:** Sanitize input trước khi truyền vào \$regex:

javascript

```
const escapeRegex = (text) => text.replace(/[-
[\]{}()*+?.,\\^$|#\s]/g, '\\$&');
const keyword = escapeRegex(req.query.search);
Product.find({ name: { $regex: keyword, $options: 'i' } });
```

3. Hạn chế của regex:

- Regex không phù hợp với tìm kiếm ngôn ngữ tự nhiên phức tạp hoặc tìm kiếm theo ngữ nghĩa. Trong các trường hợp này, cần sử dụng các công cụ như Elasticsearch.
- **Giải pháp:** Đánh giá yêu cầu dự án để quyết định khi nào nên dùng regex và khi nào cần công cụ khác.

Mẹo (Tips)

1. Kết hợp với các bộ lọc:

- Kết hợp regex với các toán tử MongoDB khác để tăng độ chính xác:

javascript

```
Product.find({
  $and: [
    { name: { $regex: 'iphon', $options: 'i' } },
    { category: 'electronics' }
  ]
});
```

2. Sử dụng text index thay thế:

- Nếu ứng dụng yêu cầu tìm kiếm văn bản thường xuyên, hãy chuyển sang text index để tăng hiệu suất:

javascript

```
Product.find({ $text: { $search: 'iphone samsung' } }).sort({
  score: { $meta: 'textScore' } });
```

3. Hỗ trợ autocomplete:

- Sử dụng regex để xây dựng tính năng gợi ý tìm kiếm:

javascript

```
app.get('/autocomplete', async (req, res) => {
  const keyword = req.query.q;
  const suggestions = await Product.find({
    name: { $regex: `^${keyword}`, $options: 'i' }
  }).limit(5).select('name');
  res.json(suggestions);
});
```

4. Kiểm tra từ khóa trước:

- Tránh chạy regex nếu từ khóa rỗng hoặc không hợp lệ:

javascript

```
if (!keyword || keyword.length < 2) {
```

```

    return res.status(400).json({ message: 'Keyword too short' });
  }
}

```

5. Sử dụng biến động:

- Cho phép người dùng tùy chỉnh cách tìm kiếm (ví dụ: khớp chính xác, chứa, bắt đầu bằng):
javascript

```

const type = req.query.type; // 'exact', 'contains', 'startsWith'
let regexPattern;
if (type === 'startsWith') regexPattern = `^${keyword}`;
else if (type === 'exact') regexPattern = `^${keyword}$`;
else regexPattern = keyword;
Product.find({ name: { $regex: regexPattern, $options: 'i' } });

```

3. Develop a Full-Fledged REST API Server with Express, MongoDB, and Mongoose

Kinh nghiệm thực tiễn

- Tính mở rộng:** REST API là nền tảng cho hầu hết các ứng dụng web và di động hiện đại. Một API được thiết kế tốt có thể phục vụ nhiều client (web, mobile, IoT) và dễ dàng mở rộng khi dự án phát triển.
- Tách biệt logic:** Trong thực tế, các dự án lớn thường tách biệt routes, controllers, models, và middleware để dễ bảo trì và làm việc nhóm. Ví dụ:
text

```

/controllers/postController.js
/routes/postRoutes.js
/models/Post.js

```

- Xác thực và phân quyền:** Hầu hết các API thực tế đều yêu cầu xác thực (authentication) và phân quyền (authorization) để bảo vệ dữ liệu. Ví dụ, chỉ người tạo bài viết mới được phép chỉnh sửa hoặc xóa bài viết.
- Ứng dụng phổ biến:**
 - Thương mại điện tử: Quản lý sản phẩm, đơn hàng, giỏ hàng.
 - Mạng xã hội: Đăng bài, bình luận, thích bài viết.
 - Ứng dụng quản lý: Quản lý công việc, dự án, hoặc tài liệu.

Vấn đề cần lưu ý

1. Bảo mật:

- Xác thực:** Sử dụng JWT, OAuth, hoặc các phương pháp xác thực khác để bảo vệ endpoint:
javascript

```

const jwt = require('jsonwebtoken');
function authMiddleware(req, res, next) {
  const token = req.header('Authorization')?.replace('Bearer ', '');
  if (!token) return res.status(401).json({ message: 'No token provided' });
  try {
    const decoded = jwt.verify(token, 'secret_key');
    req.user = decoded;
    next();
  } catch (err) {
    res.status(401).json({ message: 'Invalid token' });
  }
}

```

- NoSQL Injection:** Validate dữ liệu đầu vào để tránh các cuộc tấn công NoSQL injection. Sử dụng thư viện như Joi hoặc Mongoose validation:

javascript

```
const Joi = require('joi');
const postSchema = Joi.object({
  title: Joi.string().min(3).required(),
  content: Joi.string().min(10).required(),
  authorId: Joi.string().hex().length(24).required()
});
```

- **HTTPS:** Đảm bảo API chạy trên HTTPS để bảo vệ dữ liệu truyền tải.

2. Hiệu suất:

- **Phân trang (pagination):** Tránh trả về toàn bộ dữ liệu trong một lần gọi API. Sử dụng skip và limit:

javascript

```
app.get('/posts', async (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 10;
  const skip = (page - 1) * limit;
  try {
    const posts = await
    Post.find().skip(skip).limit(limit).populate('author');
    res.json(posts);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

- **Caching:** Sử dụng Redis hoặc caching trong bộ nhớ để lưu trữ các truy vấn thường xuyên.
- **Index:** Tạo index cho các trường thường xuyên truy vấn để tăng tốc độ:

javascript

```
postSchema.index({ title: 1, createdAt: -1 });
```

3. Xử lý lỗi:

- Luôn trả về mã trạng thái HTTP phù hợp (200, 201, 400, 404, 500) và thông báo lỗi rõ ràng:

javascript

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: 'Internal server error', error:
err.message });
});
```

- Xử lý các trường hợp đặc biệt như tài nguyên không tìm thấy (404) hoặc dữ liệu không hợp lệ (400).

4. Tài liệu API:

- Trong thực tế, các dự án lớn yêu cầu tài liệu API rõ ràng để nhóm phát triển frontend hoặc bên thứ ba sử dụng. Sử dụng Swagger hoặc Postman để tạo tài liệu tự động.

5. Versioning:

- Khi API phát triển, hãy sử dụng versioning (ví dụ: /api/v1/posts) để tránh phá vỡ các client hiện tại khi thêm tính năng mới.

Mẹo (Tips)

1. Cấu trúc code rõ ràng:

- Tách biệt logic thành các file riêng (routes, controllers, models) để dễ bảo trì:

javascript

```
// routes/postRoutes.js
const express = require('express');
const router = express.Router();
const postController = require('../controllers/postController');

router.get('/', postController.getPosts);
router.post('/', postController.createPost);

module.exports = router;

// controllers/postController.js
exports.getPosts = async (req, res) => {
  try {
    const posts = await Post.find().populate('author', 'name');
    res.json(posts);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};
```

2. Sử dụng middleware để tái sử dụng logic:

- Tạo middleware để xử lý xác thực, logging, hoặc validation: javascript

```
const validatePost = (req, res, next) => {
  if (!req.body.title || !req.body.content) {
    return res.status(400).json({ message: 'Title and content are required' });
  }
  next();
};
app.post('/posts', validatePost, authMiddleware, async (req, res) => { ... });
```

3. Tối ưu hóa truy vấn:

- Sử dụng lean() để trả về JSON thô nếu không cần các phương thức Mongoose: javascript

```
Post.find().lean().exec();
```

- Kết hợp populate và select để giảm dữ liệu trả về: javascript

```
Post.find().populate('author', 'name').select('title content').exec();
```

4. Logging và giám sát:

- Sử dụng thư viện như morgan để ghi log các yêu cầu HTTP: javascript

```
const morgan = require('morgan');
app.use(morgan('dev'));
```

- Theo dõi hiệu suất API bằng công cụ như New Relic hoặc Prometheus.

5. Test API:

- Sử dụng Postman hoặc Jest để kiểm tra các endpoint: javascript

```
const request = require('supertest');
const app = require('./app');

describe('POST /posts', () => {
  it('should create a new post', async () => {
    const res = await request(app)
```



```

        .post('/posts')
        .send({ title: 'Test', content: 'Content', authorId: '123'
    });
    expect(res.statusCode).toEqual(201);
    expect(res.body).toHaveProperty('title', 'Test');
  });
});

```

Tình huống tổng hợp thực tế

Giả sử bạn đang xây dựng một ứng dụng blog:

- **Populate:** Khi hiển thị bài viết, bạn sử dụng populate để lấy thông tin tác giả và bình luận:

javascript

```

app.get('/posts/:id', async (req, res) => {
  try {
    const post = await Post.findById(req.params.id)
      .populate('author', 'name avatar')
      .populate({
        path: 'comments',
        populate: { path: 'user', select: 'name avatar' }
      })
      .lean();
    if (!post) return res.status(404).json({ message: 'Post not found' });
    res.json(post);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

```

- **Regex:** Tạo một endpoint tìm kiếm bài viết theo từ khóa:

javascript

```

app.get('/search', async (req, res) => {
  try {
    const keyword = escapeRegex(req.query.q);
    const posts = await Post.find({
      $or: [
        { title: { $regex: keyword, $options: 'i' } },
        { content: { $regex: keyword, $options: 'i' } }
      ]
    }).populate('author', 'name').lean();
    res.json(posts);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

```

- **REST API:** Xây dựng các endpoint CRUD với xác thực:

javascript

```

app.post('/posts', authMiddleware, validatePost, async (req, res) => {
  try {
    const post = new Post({
      title: req.body.title,
      content: req.body.content,
      author: req.user.id
    });
    await post.save();
    res.status(201).json(post);
  } catch (err) {

```

```

    res.status(400).json({ message: err.message });
  }
});

```

Kết luận

- **Populate:** Tối ưu hóa truy vấn liên quan, nhưng cần cẩn thận với hiệu suất và dữ liệu null.
- **Regex:** Hữu ích cho tìm kiếm linh hoạt, nhưng cần index và sanitize input để đảm bảo hiệu suất và bảo mật.
- **REST API:** Yêu cầu thiết kế tốt về cấu trúc, bảo mật, và xử lý lỗi để tạo ra API mạnh mẽ, dễ mở rộng.

EXERCISE

TÓM TẮT DỰ ÁN

1. Thông tin tổng quan

- **Tên dự án:** Employee Management System
- **Mục tiêu:** Tạo RESTful API theo mô hình MVC cho quản lý nhân sự (CRUD hoàn chỉnh).
- **Đối tượng quản lý:** Employees, Departments, Projects, Dependents.

2. Mục tiêu học tập

- Hiểu và triển khai kiến trúc MVC trong Node.js.
- Tạo API chuẩn REST cho CRUD.
- Kết nối MongoDB với Mongoose.
- Thực hành tổ chức mã nguồn chuyên nghiệp.
- Áp dụng kỹ năng thực tiễn cho dự án backend hoàn chỉnh.

3. Công nghệ sử dụng

Thành phần	Công nghệ
Backend	Node.js, Express.js, MongoDB, Mongoose
Tools	Postman, VS Code, MongoDB Compass
Middleware	dotenv, cors, body-parser

4. Cấu trúc thư mục

pgsql

```

employee-management-system/
├── backend/
│   ├── models/
│   │   ├── Employee.js
│   │   ├── Department.js
│   │   ├── Project.js
│   │   └── Dependent.js
│   └── controllers/

```

```

├── employeeController.js
├── routes/
│   └── employeeRoutes.js
├── config/
│   └── db.js
├── .env
├── server.js
└── README.md

```

5. TÓM TẮT CÁC BƯỚC THỰC HIỆN

Backend (Node.js)

1. Cài đặt công cụ & khởi tạo dự án Node.js.
2. Cài thư viện cần thiết (express, mongoose, cors, dotenv).
3. Kết nối MongoDB (file db.js).
4. Tạo Models cho: Employee, Department, Project, Dependent.
5. Tạo Controllers: CRUD logic cho Employee.
6. Tạo Routes: mapping URL → controller.
7. Cấu hình server (server.js).
8. Test API bằng Postman.

6. CHI TIẾT TỪNG BƯỚC

Bước 1: Cài đặt

bash

```

mkdir employee-management-system
cd employee-management-system
mkdir backend && cd backend
npm init -y
npm install express mongoose dotenv cors

```

Bước 2: Cấu hình môi trường

File .env

env

```

PORT=8888
MONGO_URI=mongodb://localhost:27017/EmpDatabase

```

Bước 3: Kết nối MongoDB

File: config/db.js

js

```

const mongoose = require('mongoose');
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB connected");
  } catch (err) {
    console.error(err.message);
    process.exit(1);
  }
};
module.exports = connectDB;

```

Bước 4: Tạo Mongoose Models

models/Employee.js

js

```

const mongoose = require('mongoose');

```

```
const employeeSchema = new mongoose.Schema({
  firstName: String,
  lastName: String,
  email: String,
  phone: String,
  hireDate: Date,
  salary: Number,
  department: { type: mongoose.Schema.Types.ObjectId, ref: 'Department' },
  projects: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Project' }]
});

module.exports = mongoose.model('Employee', employeeSchema);
(Tạo tương tự cho Department, Project, Dependent)
```

◆ Bước 5: Tạo Controllers

controllers/employeeController.js

js

```
const Employee = require('../models/Employee');

exports.getAllEmployees = async (req, res) => {
  try {
    const employees = await Employee.find()
      .populate('department')
      .populate('projects');
    res.json(employees);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};

exports.createEmployee = async (req, res) => {
  try {
    const emp = new Employee(req.body);
    await emp.save();
    res.status(201).json(emp);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
};

exports.updateEmployee = async (req, res) => {
  try {
    const emp = await Employee.findByIdAndUpdate(req.params.id, req.body, {
      new: true });
    if (!emp) return res.status(404).json({ message: "Not found" });
    res.json(emp);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
};

exports.deleteEmployee = async (req, res) => {
  try {
    await Employee.findByIdAndDelete(req.params.id);
    res.json({ message: "Employee deleted" });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

◆ Bước 6: Tạo Routes

routes/employeeRoutes.js

js

```
const express = require('express');
const router = express.Router();
const employeeController = require('../controllers/employeeController');

router.get('/', employeeController.getAllEmployees);
router.post('/', employeeController.createEmployee);
router.put('/:id', employeeController.updateEmployee);
router.delete('/:id', employeeController.deleteEmployee);

module.exports = router;
```

◆ Bước 7: Tạo server

server.js

js

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const connectDB = require('./config/db');

dotenv.config();
connectDB();

const app = express();
app.use(cors());
app.use(express.json());

app.use('/employees', require('./routes/employeeRoutes'));

app.listen(process.env.PORT, () => {
  console.log(`Server is running on port ${process.env.PORT}`);
});
```

7. TEST CASES (Postman)

TC	Hành động	API	Kết quả mong đợi
TC01	Lấy danh sách nhân viên	GET /employees	Trả về danh sách JSON
TC02	Tạo nhân viên mới	POST /employees	Trả về nhân viên vừa tạo
TC03	Cập nhật nhân viên	PUT /employees/:id	Trả về bản ghi đã cập nhật
TC04	Xoá nhân viên	DELETE /employees/:id	Trả về thông báo đã xoá
TC05	Gửi POST sai định dạng	POST /employees	Trả về lỗi 400

8. LỖI THƯỜNG GẶP & Mẹo

! Lỗi phổ biến:

- ✗ **MongoError: E11000** → Lỗi trùng email → nên thêm `unique: true`.
- ✗ **populate null** → Quên `ref` trong schema hoặc chưa `require()` model.
- ✗ **Error: PORT already in use** → Đổi PORT hoặc kill port: `npx kill-port 8888`.

✓ Mẹo:

- Tạo middleware xử lý lỗi trung tâm để dùng lại (`errorHandler.js`)
- Dùng `mongoose.set('strictQuery', false)` để giảm cảnh báo.
- Khi cần seed dữ liệu: tạo file `seed.js` gọi `new Model(data).save()` hàng loạt.

✓ TỔNG KẾT

Dự án này giúp bạn:

- Làm chủ mô hình MVC.
- Thành thạo Mongoose CRUD.
- Tạo RESTful API chuyên nghiệp.
- Hiểu sâu tổ chức backend chuyên nghiệp.

PHÂN TÍCH QUY LUẬT THỰC THI

PROMT: Tôi là người mới học, tôi cần phát hiện quy luật thực thi của chương trình. Hãy Mô tả quá trình thực thi của code của bài tập

Chúng ta sẽ mô tả quá trình thực thi của chương trình theo **thứ tự dòng chảy**, từ lúc chạy `node server.js` đến khi người dùng gọi API như `GET /api/tasks`.

LUYỆN TẬP

 Bài Tập 1: Quản lý Nhân Viên (Basic CRUD)



Mục tiêu:

Tạo API CRUD cơ bản cho nhân viên.



Chức năng:

- Tạo, đọc, sửa, xóa nhân viên.
- Dữ liệu: tên, email, ngày tuyển dụng, lương.

✂ Công nghệ:

Node.js, Express, MongoDB, Mongoose, Postman



Cấu trúc:

bash

```
/employee-crud/  
├── models/Employee.js  
├── routes/employeeRoutes.js  
├── controllers/employeeController.js  
├── server.js  
└── .env
```



Bước thực hiện:

1. Tạo project, cài đặt thư viện.
2. Tạo model Employee.
3. Viết controller CRUD.
4. Tạo route và test bằng Postman.



Test Case:

TC	API	Kết quả
TC01	GET /employees	Trả về danh sách nhân viên
TC02	POST /employees	Tạo mới nhân viên
TC03	DELETE /employees/:id	Xóa nhân viên theo ID

Bài Tập 2: Quản lý Phòng Ban (Department)



Mục tiêu:

Thêm chức năng quản lý phòng ban.



Chức năng:

- Tạo, xem, cập nhật, xóa phòng ban.
- Mỗi nhân viên thuộc 1 phòng ban.



Công nghệ:

Node.js, Express, MongoDB, Mongoose



Cấu trúc:

bash

```
/department-management/  
├── models/Department.js  
├── models/Employee.js  
├── controllers/departmentController.js  
├── routes/departmentRoutes.js  
├── server.js  
└── .env
```



Bước thực hiện:

1. Tạo model Department.
2. Liên kết với Employee.
3. Tạo controller, route cho Department.



Test Case:

TC	API	Kết quả
TC01	GET /departments	Danh sách phòng ban
TC02	POST /departments	Thêm phòng ban
TC03	GET /employees → Kiểm tra populate phòng ban	

Bài Tập 3: Quản lý Dự Án (Many-to-Many)



Mục tiêu:

Quản lý mối quan hệ nhiều-nhiều giữa nhân viên và dự án.



Chức năng:

- Nhân viên có thể tham gia nhiều dự án.
- Dự án có nhiều nhân viên.



Công nghệ:

Node.js, Express, MongoDB, Mongoose



Cấu trúc:

bash

```
/project-management/  
├── models/Project.js  
├── models/Employee.js  
├── controllers/projectController.js  
├── routes/projectRoutes.js  
├── server.js  
└── .env
```



Bước thực hiện:

1. Tạo model Project (employees là mảng ObjectId).
2. Cập nhật model Employee để chứa projects.
3. Tạo API: thêm dự án, cập nhật nhân viên trong dự án.



Test Case:

TC	API	Kết quả
TC01	POST /projects	Tạo dự án
TC02	PUT /projects/:id/employees	Gán nhân viên vào dự án
TC03	GET /employees → Kiểm tra dự án được populate	

Bài Tập 4: Người Phụ Thuộc (One-to-Many)

Mục tiêu:

Quản lý người phụ thuộc của mỗi nhân viên.

Chức năng:

- Một nhân viên có nhiều người phụ thuộc.

Công nghệ:

Node.js, Express, MongoDB, Mongoose

Cấu trúc:

bash

```
/dependent-management/
├── models/Dependent.js
├── models/Employee.js
├── controllers/dependentController.js
├── routes/dependentRoutes.js
├── server.js
└── .env
```

Bước thực hiện:

1. Tạo model Dependent (liên kết với Employee).
2. Tạo API lấy người phụ thuộc theo ID nhân viên.
3. Tạo chức năng thêm người phụ thuộc.

Test Case:

TC	API	Kết quả
TC01	GET /employees/:id/dependents	Trả về người phụ thuộc
TC02	POST /dependents	Tạo người phụ thuộc
TC03	DELETE /employees/:id → Kiểm tra người phụ thuộc cũng bị xóa	

Bài Tập 5: Giao diện hiển thị nhân viên (Frontend)

Mục tiêu:

Tạo giao diện React để hiển thị danh sách nhân viên từ API.

Chức năng:

- Hiển thị danh sách nhân viên.
- Gọi API từ backend.
- Hiển thị avatar, tên, email, phòng ban.

Công nghệ:

React.js, Axios, React Hooks

Cấu trúc:

bash

```
/emp-frontend/
├── src/
│   ├── components/EmployeeList.js
│   ├── App.js
│   └── .env
```

Bước thực hiện:

1. Tạo React App: `npx create-react-app emp-frontend`

2. Tạo `.env` chứa API URL.
3. Viết component `EmployeeList` sử dụng `axios`.



Test Case:

TC	Giao diện	Kết quả
TC01	App chạy	Hiển thị danh sách nhân viên
TC02	Không có dữ liệu	Hiện thông báo "Không có nhân viên"
TC03	Avatar lỗi	Hiển thị ảnh mặc định